Library Holdings & E-book Automation Tool

A Project Summary

July 15, 2026

A Note on Scope: This is a student-built project, not a finished library product, and it isn't perfect. It was built around one librarian's weeding process (Lisa Morrison) and the specific pieces of Primo data she uses to make those calls, and every librarian tends to weed a little differently. It'll also occasionally misfire on messy titles, unusual formatting, or edge cases that just weren't covered yet, though I did my best to try cover as many as I could think of/find. Treat this as a working prototype that saves time on one specific workflow, not a plug-and-play tool ready for library-wide rollout.

Overview

This tool takes something that used to be done completely by hand and automates it, for a big list of book titles, it checks Primo to see how many other schools have a copy of each one, and whether there's an e-book version available. A librarian uploads a spreadsheet of titles, anywhere from a few dozen up to a few thousand, picks the row range they want done, and the tool searches every single one in Primo, makes sure it actually found the right book and not just something similar, counts up how many schools hold it, checks for an ebook, and hands back a finished spreadsheet with everything filled in and color coded. I am planning on hosting the server off a rasberry pi in the library so anyone on staff can run it from the browser.

How It's Built

It's a small Flask web app. You go to the page, built with plain HTML, CSS, and vanilla JavaScript so nothing fancy is required to run it, upload a `.xlsx` file, and pick a start and end row. That form gets submitted to a `/stream` route in `app.py`, which saves the file and kicks off the job as a live streaming connection so the browser shows progress. If someone closes the tab mid-run the backend notices and cleans everything up on its own.

The actual work happens in `first.py`, which uses Playwright to control a headless Chromium browser and click through Primo basically the same way a person would. For each row it splits the "TITLE" column into a title and author, strips out junk like "Sir," "Edited by," Roman numerals, that kind of thing, and cleans up any special characters

so accented names don't break the search. Then it builds a Primo search URL and opens whatever comes back as the top result.

Here's the part that took the most work and is honestly the core of the whole project, figuring out whether that top result is actually the right book. It's not just matching a keyword. Two checks have to both pass. A title check, which needs roughly 80% word overlap between the sheet's title and what's on the catalog page since titles rarely match word for word once you account for subtitles and formatting. And an author check, which pulls out anything that looks like a name from both sides, tries to match full names and initials against each other, and if there's exactly one name left over on each side it runs a fuzzy match to catch typos or spelling differences. A book only counts as confirmed if at least one full name lines up and enough of the name pieces match overall.

Once it's confirmed, it counts how many other schools have that record and checks for an ebook, then follows Primo's linked records for that title (one book can be split across multiple catalog entries) and does the same check on each one, adding up the school count as it goes. If a search comes back empty, or wrong, or there's no author to search on, or something breaks, it doesn't guess, it just logs why, NO RESULTS, BAD SEARCH, NO AUTHOR, ERROR, so those rows can get looked at by hand later. It'll also retry once automatically before giving up on a title. Every result gets written straight into the spreadsheet the moment it's found using openpyxl, so if something crashes halfway through you don't lose the work already done, and progress gets sent back to the browser after every book.

Once the whole range is done, `format.py` goes through and color codes the results column, green for good matches, different colors for each kind of flag, no results, no author, bad search, error, N/A, so whoever's reviewing it can see right away which rows need a second look. Now this script is not without its faults, and I am not claiming that it works perfectly. It is also tailored to Lisa Morrison's method of weeding, and it seems every librarian has their own unique way of decideing to weed and what information they want from primo link to determine that.

Stack: Python, Flask, Playwright for the browser automation, openpyxl for reading/writing/formatting the spreadsheet, pandas, plain HTML/CSS/JS for the front end, Server-Sent Events for the live progress bar.